

```
#####
#
# COURS : GÉOMÉTRIE DANS LE PLAN ET DANS L'ESPACE
# Théorie · Démonstrations · Code Python natif
#
#####
#
# COMMENT LIRE CE COURS ?
#
# Même structure que le cours sur le raisonnement mathématique :
# [THÉORIE]      – L'idée, pourquoi ça existe
# [DÉFINITION]   – La règle formelle
# [DÉMONSTRATION] – Preuve pas à pas
# [EXEMPLE]      – Exemple concret
# [CODE PYTHON]  – Programme qui illustre la notion
# [PIÈGE]       – Erreur classique à éviter
#
# Analogie directrice :
# Le plan = une feuille de papier infinie → 2 coordonnées (x, y)
# L'espace = le monde physique → 3 coordonnées (x, y, z)
#
#####
```

PARTIE 1 – FONDATIONS : POINTS, VECTEURS, COORDONNÉES

1. POINTS ET REPÈRES – se situer dans l'espace

[THÉORIE]

Un point est une position sans dimension. Comme une adresse GPS : elle repère un endroit précis, elle n'a ni longueur ni aire. Pour travailler avec des points on choisit un REPÈRE : une origine 0 et des axes perpendiculaires gradués.

Plan (2D) : repère (0, x, y) → chaque point = (x, y)

Espace (3D) : repère (0, x, y, z) → chaque point = (x, y, z)

[DÉFINITION]

Dans un repère cartésien orthonormé (0, i, j) du plan :

Tout point M est repéré par ses coordonnées M(x, y).

0 est l'origine, de coordonnées (0, 0).

i = (1, 0) : vecteur unitaire de l'axe des abscisses (axe x)

j = (0, 1) : vecteur unitaire de l'axe des ordonnées (axe y)

En espace : repère (0, i, j, k) avec k = (0, 0, 1) axe des cotes (axe z).

[EXEMPLE]

A(2, 3) : à 2 unités à droite de 0, 3 en haut
B(-1, 4) : à 1 unité à gauche, 4 en haut
C(0, 0) : l'origine
P(1, 2, 3): dans l'espace, 1 selon x, 2 selon y, 3 selon z

[CODE PYTHON]

```
import math

class Point2D:
    def __init__(self, x, y):
        self.x = x
        self.y = y
    def __repr__(self):
        return f"Point({self.x}, {self.y})"

class Point3D:
    def __init__(self, x, y, z):
        self.x = x
        self.y = y
        self.z = z
    def __repr__(self):
        return f"Point({self.x}, {self.y}, {self.z})"

A = Point2D(2, 3)
B = Point2D(-1, 4)
P = Point3D(1, 2, 3)

print("A =", A)    # Point(2, 3)
print("B =", B)    # Point(-1, 4)
print("P =", P)    # Point(1, 2, 3)

dist_A_origine = math.sqrt(A.x**2 + A.y**2)
print(f"|OA| = {dist_A_origine:.4f}")    # 3.6056
```

2. VECTEURS – décrire un déplacement

[THÉORIE]

Un point dit "où on est". Un vecteur dit "comment on se déplace".
C'est comme la différence entre une adresse et un itinéraire.
Un vecteur est une flèche : il a une DIRECTION, un SENS et une LONGUEUR (norme). Sa position de départ n'a pas d'importance : deux flèches parallèles de même longueur et même sens sont le MÊME vecteur (on dit qu'ils sont égaux).

[DÉFINITION]

Le vecteur AB est défini par deux points A et B :

$AB = (x_B - x_A, y_B - y_A)$ en 2D
 $AB = (x_B - x_A, y_B - y_A, z_B - z_A)$ en 3D

Norme (longueur) :

$||AB|| = \sqrt{(x_B - x_A)^2 + (y_B - y_A)^2}$ en 2D

$||AB|| = \sqrt{(x_B - x_A)^2 + (y_B - y_A)^2 + (z_B - z_A)^2}$ en 3D

Opérations sur les vecteurs $u(a, b)$ et $v(c, d)$:

Addition : $u + v = (a+c, b+d)$

Soustraction : $u - v = (a-c, b-d)$

Multiplication scalaire : $\lambda u = (\lambda a, \lambda b)$

Vecteur nul : $0 = (0, 0)$

Règle de Chasles : $AB + BC = AC$

(enchaîner deux déplacements = déplacement global de A vers C)

[DÉMONSTRATION] de la règle de Chasles en coordonnées

Soient $A(x_A, y_A)$, $B(x_B, y_B)$, $C(x_C, y_C)$.

$AB = (x_B - x_A, y_B - y_A)$

$BC = (x_C - x_B, y_C - y_B)$

$AB + BC = (x_B - x_A + x_C - x_B, y_B - y_A + y_C - y_B)$
 $= (x_C - x_A, y_C - y_A)$
 $= AC \quad \square$

[CODE PYTHON]

```
import math
```

```
class Vecteur2D:
    def __init__(self, x, y):
        self.x = x
        self.y = y
    def __add__(self, o):
        return Vecteur2D(self.x + o.x, self.y + o.y)
    def __sub__(self, o):
        return Vecteur2D(self.x - o.x, self.y - o.y)
    def __mul__(self, k):
        return Vecteur2D(k * self.x, k * self.y)
    def norme(self):
        return math.sqrt(self.x**2 + self.y**2)
    def __repr__(self):
        return f"({self.x}, {self.y})"
```

```
def vecteur_AB(A, B):
    return Vecteur2D(B.x - A.x, B.y - A.y)
```

```
A = Point2D(1, 2)
```

```
B = Point2D(4, 6)
```

```
C = Point2D(7, 3)
```

```
AB = vecteur_AB(A, B)    # (3, 4)
```

```
BC = vecteur_AB(B, C)    # (3, -3)
```

```
AC = vecteur_AB(A, C)    # (6, 1)
```

```
print("AB =", AB)
print("BC =", BC)
print("AB + BC =", AB + BC)    # (6, 1) = AC → Chasles ✓
print("||AB|| =", AB.norme()) # 5.0
```

[PIÈGE]

Un point A(2, 3) et un vecteur v(2, 3) ne sont PAS la même chose.
Le point est une POSITION. Le vecteur est un DÉPLACEMENT.
Même écriture, objets différents – distinguez-les toujours.

3. PRODUIT SCALAIRE – mesurer l'alignement entre deux vecteurs

[THÉORIE]

Comment savoir si deux vecteurs "vont dans la même direction" ?
Le produit scalaire répond à ça. C'est un NOMBRE (pas un vecteur)
qui mesure à quel point deux vecteurs sont alignés.
Il est fondamental pour : les angles, la perpendicularité,
les projections, et en data science pour la similarité cosinus
(NLP, recommandation, réseaux de neurones...).

[DÉFINITION]

Produit scalaire en 2D :

Formule algébrique : $u(a,b) \cdot v(c,d) = ac + bd$

Formule géométrique : $u \cdot v = ||u|| * ||v|| * \cos(\theta)$

où θ est l'angle entre u et v.

Produit scalaire en 3D :

$u(a,b,c) \cdot v(d,e,f) = ad + be + cf$

Propriétés clés :

$u \cdot v = 0 \iff u$ perpendiculaire à v

$u \cdot u = ||u||^2$

Symétrique : $u \cdot v = v \cdot u$

Bilinéaire : $(u + v) \cdot w = u \cdot w + v \cdot w$

[DÉMONSTRATION] Lien entre les deux formules du produit scalaire

Dans le triangle formé par u, v et (v - u), la loi des cosinus :

$||v - u||^2 = ||u||^2 + ||v||^2 - 2*||u||*||v||*\cos(\theta)$

Or par développement algébrique :

$||v - u||^2 = (v-u) \cdot (v-u) = ||v||^2 - 2(u \cdot v) + ||u||^2$

En identifiant les deux expressions :

$-2(u \cdot v) = -2*||u||*||v||*\cos(\theta)$

$u \cdot v = ||u||*||v||*\cos(\theta) \quad \square$

[EXEMPLE]

$u = (1, 0), v = (0, 1) \rightarrow u \cdot v = 1*0 + 0*1 = 0 \rightarrow$ perpendiculaires ✓

$u = (3, 4), v = (3, 4) \rightarrow u \cdot v = 9 + 16 = 25 = ||u||^2$ ✓

$u = (1, 0), v = (1, 1) \rightarrow \cos(\theta) = 1 / (1 * \sqrt{2}) \rightarrow \theta = 45^\circ$

[CODE PYTHON]

```
import math

def produit_scalaire_2d(u, v):
    return u.x * v.x + u.y * v.y

def produit_scalaire_3d(u, v):
    return u[0]*v[0] + u[1]*v[1] + u[2]*v[2]

def angle_entre(u, v):
    """Angle en degrés entre deux vecteurs 2D"""
    ps = produit_scalaire_2d(u, v)
    cos_t = ps / (u.norme() * v.norme())
    cos_t = max(-1, min(1, cos_t)) # sécuriser les arrondis flottants
    return math.degrees(math.acos(cos_t))

u = Vecteur2D(1, 0)
v = Vecteur2D(0, 1)
w = Vecteur2D(1, 1)

print("u · v =", produit_scalaire_2d(u, v)) # 0 → perpendiculaires
print("u · w =", produit_scalaire_2d(u, w)) # 1
print("Angle u,v :", angle_entre(u, v), "°") # 90.0 °
print("Angle u,w :", angle_entre(u, w), "°") # 45.0 °

# Vérification : u perpendiculaire à v <=> u·v = 0
def sont_perpendiculaires(u, v, tol=1e-9):
    return abs(produit_scalaire_2d(u, v)) < tol

print("u perp v ?", sont_perpendiculaires(u, v)) # True
print("u perp w ?", sont_perpendiculaires(u, w)) # False
```

4. PRODUIT VECTORIEL – créer un vecteur perpendiculaire (3D uniquement)

[THÉORIE]

Le produit vectoriel n'existe qu'en 3D. Il prend deux vecteurs et produit un TROISIÈME vecteur perpendiculaire aux deux premiers. C'est l'outil pour : trouver la normale à un plan, calculer une aire, détecter si trois points sont alignés ou non.

[DÉFINITION]

$u(a, b, c) \times v(d, e, f) = (bf - ce, cd - af, ae - bd)$

Propriétés :

$u \times v$ est perpendiculaire à u ET à v $(u \times v) \cdot u = 0, (u \times v) \cdot v = 0$
 $||u \times v|| = ||u|| * ||v|| * \sin(\theta)$
 $u \times v = 0 \iff u$ et v sont colinéaires (même direction)

Anticommutativité : $u \times v = -(v \times u)$

[EXEMPLE]

$i = (1,0,0)$, $j = (0,1,0)$

$i \times j = (0*0-0*1, 0*0-1*0, 1*1-0*0) = (0, 0, 1) = k$ ✓

(le produit des deux premiers vecteurs de base donne le troisième)

[DÉMONSTRATION] $(u \times v)$ est perpendiculaire à u

Calculons $(u \times v) \cdot u$ avec $u = (a,b,c)$, $v = (d,e,f)$:

$u \times v = (bf-ce, cd-af, ae-bd)$

$(u \times v) \cdot u = a(bf-ce) + b(cd-af) + c(ae-bd)$
 $= abf - ace + bcd - abf + ace - bcd$
 $= 0 \quad \square$

[CODE PYTHON]

```
def produit_vectoriel(u, v):
    """u et v sont des tuples (a, b, c)"""
    a, b, c = u
    d, e, f = v
    return (b*f - c*e, c*d - a*f, a*e - b*d)

def norme3d(v):
    return math.sqrt(v[0]**2 + v[1]**2 + v[2]**2)

i = (1, 0, 0)
j = (0, 1, 0)
k = (0, 0, 1)

print("i x j =", produit_vectoriel(i, j))    # (0, 0, 1) = k ✓
print("j x i =", produit_vectoriel(j, i))    # (0, 0, -1) anticommutatif ✓

# Vérifier la perpendicularité
u = (1, 2, 3)
v = (4, 5, 6)
w = produit_vectoriel(u, v)
print("\nu x v =", w)
print("(u x v) · u =", produit_scalaire_3d(w, u))  # 0 ✓
print("(u x v) · v =", produit_scalaire_3d(w, v))  # 0 ✓

# Aire du parallélogramme formé par u et v
aire = norme3d(produit_vectoriel(u, v))
print(f"Aire parallélogramme (u,v) = {aire:.4f}")
```

[PIÈGE]

Le produit vectoriel $u \times v$ n'est PAS le même que $v \times u$.
Il change de signe (anticommutatif). En pratique : pour trouver
la "bonne" normale à un plan, l'ordre des vecteurs compte.

5. DISTANCE ET MILIEU – mesurer et partager

[DÉFINITION]

Distance entre deux points $A(x_A, y_A)$ et $B(x_B, y_B)$:

$$d(A, B) = ||AB|| = \sqrt{(x_B - x_A)^2 + (y_B - y_A)^2}$$

Milieu I du segment $[AB]$:

$$I = ((x_A + x_B)/2, (y_A + y_B)/2)$$

En 3D :

$$d(A, B) = \sqrt{(x_B - x_A)^2 + (y_B - y_A)^2 + (z_B - z_A)^2}$$

$$\text{Milieu} = ((x_A + x_B)/2, (y_A + y_B)/2, (z_A + z_B)/2)$$

[DÉMONSTRATION] Théorème de Pythagore → formule de distance

Dans le triangle rectangle formé par A , B et le point $C(x_B, y_A)$:

$AC = |x_B - x_A|$ (horizontal), $CB = |y_B - y_A|$ (vertical)

Par Pythagore : $AB^2 = AC^2 + CB^2 = (x_B - x_A)^2 + (y_B - y_A)^2$

Donc $AB = \sqrt{(x_B - x_A)^2 + (y_B - y_A)^2}$ □

[CODE PYTHON]

```
def distance_2d(A, B):
    return math.sqrt((B.x - A.x)**2 + (B.y - A.y)**2)

def milieu_2d(A, B):
    return Point2D((A.x + B.x) / 2, (A.y + B.y) / 2)

A = Point2D(1, 2)
B = Point2D(4, 6)
I = milieu_2d(A, B)

print(f"d(A,B) = {distance_2d(A, B)}")    # 5.0
print(f"Milieu = {I}")                    # Point(2.5, 4.0)

# Vérification : I est bien à égale distance de A et B
print(f"d(A,I) = {distance_2d(A, I):.4f}") # 2.5
print(f"d(I,B) = {distance_2d(I, B):.4f}") # 2.5 ✓
```

6. DROITES DANS LE PLAN – équations et positions relatives

[THÉORIE]

Une droite dans le plan, c'est un ensemble infini de points alignés.

On peut la décrire de trois façons différentes selon le contexte :

équation cartésienne, équation réduite, ou représentation paramétrique.
Chaque forme a ses avantages – comme choisir entre SQL et pandas
selon la tâche.

[DÉFINITION]

FORME CARTÉSIENNE (générale) :

$$ax + by + c = 0 \quad \text{avec } (a, b) \neq (0, 0)$$

Exemples : $2x - 3y + 6 = 0$, $x = 4$ (droite verticale), $y = -1$

FORME RÉDUITE (pente-ordonnée) :

$$y = mx + p$$

m = pente = $\tan(\text{angle avec l'axe } x)$ (impossible si droite verticale)

p = ordonnée à l'origine (valeur de y en $x=0$)

FORME PARAMÉTRIQUE :

Point $A(x_A, y_A)$ sur la droite + vecteur directeur $d(a, b)$:

$$\{ x = x_A + t \cdot a$$

$$\{ y = y_A + t \cdot b \quad t \in \mathbb{R}$$

POSITIONS RELATIVES de deux droites d_1 et d_2 :

Sécantes : elles se croisent en un seul point (pentes différentes)

Parallèles : même pente, ordonnées différentes (pas de croisement)

Confondues : même droite (même pente, même ordonnée)

Perpendiculaires : $m_1 \cdot m_2 = -1$

[DÉMONSTRATION] Condition de perpendicularité $m_1 \cdot m_2 = -1$

$$d_1 : y = m_1 x + p_1 \quad \text{vecteur directeur } u_1 = (1, m_1)$$

$$d_2 : y = m_2 x + p_2 \quad \text{vecteur directeur } u_2 = (1, m_2)$$

$$d_1 \perp d_2 \iff u_1 \cdot u_2 = 0$$

$$\iff 1 \cdot 1 + m_1 \cdot m_2 = 0$$

$$\iff m_1 \cdot m_2 = -1 \quad \square$$

[EXEMPLE]

$$d_1 : y = 2x + 1 \quad (\text{pente } 2)$$

$$d_2 : y = -x/2 + 3 \quad (\text{pente } -1/2)$$

$$m_1 \cdot m_2 = 2 \cdot (-1/2) = -1 \rightarrow \text{perpendiculaires } \checkmark$$

[CODE PYTHON]

```
class Droite2D:
    """Droite ax + by + c = 0"""
    def __init__(self, a, b, c):
        self.a = a
        self.b = b
        self.c = c

    def pente(self):
        if self.b == 0:
            return None # droite verticale
        return -self.a / self.b

    def ordonnee_origine(self):
        if self.b == 0:
```



```

        return None

    return -self.c / self.b

def contient(self, P):
    return abs(self.a*P.x + self.b*P.y + self.c) < 1e-9

def intersection(self, other):
    """Intersection de deux droites (ou None si parallèles)"""
    det = self.a * other.b - other.a * self.b
    if abs(det) < 1e-9:
        return None # parallèles ou confondues
    x = (-self.c * other.b + other.c * self.b) / det
    y = (-self.a * other.c + other.a * self.c) / det
    return Point2D(x, y)

def __repr__(self):
    return f"{self.a}x + {self.b}y + {self.c} = 0"

# d1 : 2x - y + 1 = 0 (y = 2x + 1)
# d2 : x + 2y - 6 = 0 (y = -x/2 + 3)
d1 = Droite2D(2, -1, 1)
d2 = Droite2D(1, 2, -6)

print("Pente d1 :", d1.pente()) # 2.0
print("Pente d2 :", d2.pente()) # -0.5
print("m1 * m2 =", d1.pente() * d2.pente()) # -1.0 → perpendiculaires ✓

I = d1.intersection(d2)
print("Intersection :", I) # Point(0.8, 2.6) environ

# Vérifier que I est sur les deux droites
print("I sur d1 ?", d1.contient(I)) # True
print("I sur d2 ?", d2.contient(I)) # True

# Distance d'un point P à une droite ax+by+c=0
def distance_point_droite(P, d):
    return abs(d.a*P.x + d.b*P.y + d.c) / math.sqrt(d.a**2 + d.b**2)

P = Point2D(3, 1)
print(f"\nDistance de P{P} à d1 : {distance_point_droite(P, d1):.4f}")

```

[PIÈGE]

La forme $y = mx + p$ n'existe pas pour les droites VERTICALES ($x = k$).
 Dans ce cas $b = 0$ dans $ax + by + c = 0$. La pente est "infinie".
 Travailler uniquement avec la forme $ax + by + c = 0$ évite ce piège.

7. TRIANGLES – propriétés fondamentales

[THÉORIE]

Le triangle est la figure géométrique la plus fondamentale du plan.

Tout polygone peut être découpé en triangles. Ses propriétés (somme des angles, théorème de Pythagore, médiatrices...) sont les briques de toute la géométrie plane.

[DÉFINITION]

Triangle ABC : trois points non alignés A, B, C.

CLASSIFICATION par les côtés :

Équilatéral : $AB = BC = CA$

Isocèle : deux côtés égaux

Scalène : trois côtés différents

CLASSIFICATION par les angles :

Rectangle : un angle de 90°

Obtusangle : un angle $> 90^\circ$

Acutangle : tous les angles $< 90^\circ$

FORMULE DE L'AIRE :

Via la base et la hauteur : $Aire = (base * hauteur) / 2$

Via les côtés (Héron) avec $s = (a+b+c)/2$:

$Aire = \sqrt{s(s-a)(s-b)(s-c)}$

Via le produit vectoriel :

$Aire = (1/2) * ||AB \times AC||$ (en 3D, projeté en 2D)

SOMME DES ANGLES : $\alpha + \beta + \gamma = 180^\circ$

[DÉMONSTRATION] Somme des angles d'un triangle = 180°

Soit ABC un triangle. Traçons par A une droite d parallèle à BC.

L'angle BAC est noté β .

Par parallélisme : $\text{angle}(d, AB) = \text{angle}(ABC) = \alpha$ (alternes-internes)

$\text{angle}(d, AC) = \text{angle}(ACB) = \gamma$ (alternes-internes)

Les angles α , β , γ forment un angle plat en A sur d :

$\alpha + \beta + \gamma = 180^\circ \quad \square$

[CODE PYTHON]

```
class Triangle:
    def __init__(self, A, B, C):
        self.A = A
        self.B = B
        self.C = C

    def cotes(self):
        a = distance_2d(self.B, self.C)
        b = distance_2d(self.A, self.C)
        c = distance_2d(self.A, self.B)
        return a, b, c

    def perimetre(self):
        return sum(self.cotes())

    def aire_heron(self):
```

```

a, b, c = self.cotes()
s = (a + b + c) / 2
return math.sqrt(s * (s-a) * (s-b) * (s-c))

def aire_vecteur(self):
    """Aire via le produit en croix 2D"""
    ab = vecteur_AB(self.A, self.B)
    ac = vecteur_AB(self.A, self.C)
    # |ab x ac| en 2D = |ab.x*ac.y - ab.y*ac.x|
    return abs(ab.x * ac.y - ab.y * ac.x) / 2

def angles_deg(self):
    a, b, c = self.cotes()
    # Loi des cosinus : cos(A) = (b^2 + c^2 - a^2) / (2bc)
    A_ang = math.degrees(math.acos((b**2 + c**2 - a**2) / (2*b*c)))
    B_ang = math.degrees(math.acos((a**2 + c**2 - b**2) / (2*a*c)))
    C_ang = 180 - A_ang - B_ang
    return A_ang, B_ang, C_ang

def type_triangle(self):
    a, b, c = sorted(self.cotes())
    if abs(a**2 + b**2 - c**2) < 1e-9:
        return "Rectangle"
    elif c**2 > a**2 + b**2:
        return "Obtusangle"
    else:
        return "Acutangle"

A = Point2D(0, 0)
B = Point2D(3, 0)
C = Point2D(0, 4)
T = Triangle(A, B, C)

print("Côtés :", T.cotes()) # (4, 5, 3) → 3-4-5
print("Périmètre :", T.perimetre()) # 12
print("Aire (Héron) :", T.aire_heron()) # 6.0
print("Aire (vecteur) :", T.aire_vecteur()) # 6.0 ✓
print("Angles :", [f"{a:.1f}°" for a in T.angles_deg()]) # [53.1°, 90.0°, 36.9°]
print("Somme angles :", sum(T.angles_deg()), "°") # 180.0 ✓
print("Type :", T.type_triangle()) # Rectangle

```

8. CERCLE – équation et propriétés

[DÉFINITION]

Cercle de centre $O(a, b)$ et de rayon r :

Ensemble des points $M(x, y)$ tels que $d(O, M) = r$.

Équation : $(x - a)^2 + (y - b)^2 = r^2$

Forme développée : $x^2 + y^2 + dx + ey + f = 0$

(on reconnaît un cercle si le coefficient de x^2 = celui de y^2)

Mesures :

Circonférence = $2\pi r$

Aire du disque = πr^2

Propriétés remarquables :

Tout angle inscrit dans un demi-cercle est droit (théorème de Thalès).

La tangente en un point M est perpendiculaire au rayon OM.

[DÉMONSTRATION] Centre et rayon depuis la forme développée

$x^2 + y^2 + dx + ey + f = 0$

Compléter le carré :

$(x + d/2)^2 - d^2/4 + (y + e/2)^2 - e^2/4 + f = 0$

$(x + d/2)^2 + (y + e/2)^2 = d^2/4 + e^2/4 - f$

Donc : centre = $(-d/2, -e/2)$, $r^2 = d^2/4 + e^2/4 - f$ □

[CODE PYTHON]

```
class Cercle:
    def __init__(self, centre, r):
        self.centre = centre
        self.r = r

    def contient(self, P):
        return abs(distance_2d(self.centre, P) - self.r) < 1e-9

    def aire(self):
        return math.pi * self.r**2

    def circonference(self):
        return 2 * math.pi * self.r

    def tangente_en(self, M):
        """Retourne la droite tangente en M (si M est sur le cercle)"""
        # Vecteur OM = vecteur normal à la tangente
        om = vecteur_AB(self.centre, M)
        # Tangente : om.x*(x - M.x) + om.y*(y - M.y) = 0
        # soit om.x*x + om.y*y - (om.x*M.x + om.y*M.y) = 0
        a = om.x
        b = om.y
        c = -(a * M.x + b * M.y)
        return Droite2D(a, b, c)

    def intersection_droite(self, d):
        """Nombre de points d'intersection avec une droite"""
        dist = distance_point_droite(self.centre, d)
        if dist > self.r + 1e-9:
            return 0, "sécante externe (pas d'intersection)"
        elif abs(dist - self.r) < 1e-9:
            return 1, "tangente"
        else:
            return 2, "sécante (2 points)"
```

```

O = Point2D(0, 0)
C = Cercle(0, 5)
P = Point2D(3, 4)

print("P sur le cercle ?", C.contient(P))      # True (3^2+4^2=25=5^2)
print("Aire :", f"{C.aire():.4f}")            # 78.5398
print("Circonférence :", f"{C.circonference():.4f}") # 31.4159

tang = C.tangente_en(P)
print("Tangente en P :", tang)                # 3x + 4y + (-25) = 0
print("P sur tangente ?", tang.contient(P))    # True ✓

# Vérifier : tangente perpendiculaire au rayon OP
d_OP = Droite2D(-P.y, P.x, 0) # direction du rayon (perpendiculaire à tang)
print("Produit pentes rayon * tangente :", end=" ")
m_tang = tang.pente()
m_rayon = -P.x / P.y if P.y != 0 else None
if m_tang and m_rayon:
    print(m_tang * m_rayon) # -1.0 ✓

n, desc = C.intersection_droite(Droite2D(1, 0, -5))
print(f"\nDroite x=5 : {n} intersection(s) → {desc}") # 1, tangente

```

PARTIE 3 – GÉOMÉTRIE DANS L'ESPACE (3D)

9. PLANS DANS L'ESPACE – l'analogue 3D de la droite

[THÉORIE]

En 3D, la droite reste une droite, mais on a un nouvel objet : le PLAN.
 Un plan c'est une surface plate infinie – l'analogue 3D de la droite 2D.
 Il est déterminé par : 3 points non alignés, ou 1 point + 1 vecteur normal.
 Le vecteur normal n est perpendiculaire à TOUS les vecteurs du plan.

[DÉFINITION]

Plan d'équation cartésienne :

$$ax + by + cz + d = 0$$

$n = (a, b, c)$ est le vecteur normal au plan.

Plan passant par $P_0(x_0, y_0, z_0)$ de vecteur normal $n(a, b, c)$:

$$a*(x - x_0) + b*(y - y_0) + c*(z - z_0) = 0$$

Forme paramétrique (avec deux vecteurs directeurs u et v non colinéaires) :

$$M = A + s*u + t*v \quad s, t \in \mathbb{R}$$

(tout point du plan s'écrit comme une combinaison de A, u, v)

POSITIONS RELATIVES de deux plans P1 et P2 :

Parallèles : normales proportionnelles, plans distincts

Confondus : même plan

Sécants : ils se croisent selon une droite

[DÉMONSTRATION] Trouver le plan passant par 3 points A, B, C

Données : A(1,0,0), B(0,1,0), C(0,0,1)

Vecteurs dans le plan : $\overrightarrow{AB} = (-1, 1, 0)$, $\overrightarrow{AC} = (-1, 0, 1)$

Vecteur normal : $\mathbf{n} = \overrightarrow{AB} \times \overrightarrow{AC}$

$$\begin{aligned}\mathbf{n} &= (1 \cdot 1 - 0 \cdot 0, \quad 0 \cdot (-1) - (-1) \cdot 1, \quad (-1) \cdot 0 - 1 \cdot (-1)) \\ &= (1, 1, 1)\end{aligned}$$

Équation : $1 \cdot (x-1) + 1 \cdot (y-0) + 1 \cdot (z-0) = 0$

$$x + y + z - 1 = 0$$

Vérification : A(1,0,0) : $1+0+0-1=0$ ✓ B(0,1,0) : $0+1+0-1=0$ ✓ □

[CODE PYTHON]

```
class Plan:
    """Plan ax + by + cz + d = 0"""
    def __init__(self, a, b, c, d):
        self.a = a
        self.b = b
        self.c = c
        self.d = d

    @classmethod
    def par_trois_points(cls, A, B, C):
        """Construit le plan passant par A, B, C"""
        AB = (B.x-A.x, B.y-A.y, B.z-A.z)
        AC = (C.x-A.x, C.y-A.y, C.z-A.z)
        n = produit_vectoriel(AB, AC) # vecteur normal
        a, b, c = n
        d = -(a*A.x + b*A.y + c*A.z)
        return cls(a, b, c, d)

    def normale(self):
        return (self.a, self.b, self.c)

    def contient(self, P):
        val = self.a*P.x + self.b*P.y + self.c*P.z + self.d
        return abs(val) < 1e-9

    def distance_point(self, P):
        num = abs(self.a*P.x + self.b*P.y + self.c*P.z + self.d)
        den = math.sqrt(self.a**2 + self.b**2 + self.c**2)
        return num / den

    def __repr__(self):
        return f"{self.a}x + {self.b}y + {self.c}z + {self.d} = 0"

# Plan passant par A(1,0,0), B(0,1,0), C(0,0,1)
A3 = Point3D(1, 0, 0)
```

```

B3 = Point3D(0, 1, 0)
C3 = Point3D(0, 0, 1)
plan = Plan.par_trois_points(A3, B3, C3)

print("Plan :", plan)                                # 1x + 1y + 1z + (-1) = 0
print("A sur plan ?", plan.contient(A3))              # True
print("B sur plan ?", plan.contient(B3))              # True
print("C sur plan ?", plan.contient(C3))              # True

P = Point3D(2, 2, 2)
print(f"\nDistance de P(2,2,2) au plan : {plan.distance_point(P):.4f}")
# (2+2+2-1)/sqrt(3) = 5/sqrt(3) ≈ 2.8868

```

10. DROITES DANS L'ESPACE – représentation paramétrique

[THÉORIE]

En 3D, une droite ne peut pas s'écrire comme une seule équation cartésienne (une équation donne un plan, pas une droite).
On utilise la forme PARAMÉTRIQUE : un point de départ + une direction.

[DÉFINITION]

Droite passant par $A(x_A, y_A, z_A)$ avec vecteur directeur $d(a, b, c)$:

$$\begin{cases} x = x_A + t \cdot a \\ y = y_A + t \cdot b \\ z = z_A + t \cdot c \end{cases} \quad t \in \mathbb{R}$$

Ou en écriture vectorielle : $M = A + t \cdot d$

POSITIONS RELATIVES de deux droites en 3D :

Sécantes : elles se croisent en un point (coplanaires)
Parallèles : même direction, pas de croisement
Confondues : même droite
Gauches : non coplanaires, ni parallèles, ni sécantes
(impossible en 2D, spécifique à la 3D !)

[CODE PYTHON]

```

class Droite3D:
    def __init__(self, point, direction):
        """point : Point3D, direction : tuple (a,b,c)"""
        self.A = point
        self.d = direction

    def point_en(self, t):
        """Retourne le point de la droite pour le paramètre t"""
        return Point3D(
            self.A.x + t * self.d[0],
            self.A.y + t * self.d[1],
            self.A.z + t * self.d[2]

```

```

)

def sont_paralleles(self, other):
    """Vérifie si deux droites ont des directions proportionnelles"""
    d1 = self.d
    d2 = other.d
    # Colinéaires si produit vectoriel nul
    pv = produit_vectoriel(d1, d2)
    return norme3d(pv) < 1e-9

def distance_a_point(self, P):
    """Distance minimale d'un point à la droite"""
    AP = (P.x - self.A.x, P.y - self.A.y, P.z - self.A.z)
    pv = produit_vectoriel(AP, self.d)
    return norme3d(pv) / norme3d(self.d)

# Droite passant par O(0,0,0) dans la direction (1,1,1)
D1 = Droite3D(Point3D(0,0,0), (1,1,1))
D2 = Droite3D(Point3D(1,0,0), (2,2,2)) # même direction → parallèle

print("D1 // D2 ?", D1.sont_paralleles(D2))    # True

P = Point3D(1, 0, 0)
print(f"Distance de P(1,0,0) à D1 : {D1.distance_a_point(P):.4f}")

# Paramétrage : points sur D1
for t in [0, 1, 2, -1]:
    print(f"  t={t} → {D1.point_en(t)}")

```

[PIÈGE]

En 3D il existe des droites GAUCHES : ni parallèles ni sécantes.
 Exemple : l'axe x et la droite passant par (0,1,0) dans la direction (0,0,1).
 Elles ne se croisent jamais et ne sont pas parallèles.
 Ce cas n'existe pas en 2D – c'est une subtilité propre à l'espace.

11. SOLIDES DE L'ESPACE – volumes et surfaces

[THÉORIE]

Les solides sont les objets 3D usuels. Pour chacun on retient
 deux formules : VOLUME (contenu) et AIRE LATÉRALE + TOTALE (enveloppe).
 Ces formules se démontrent toutes par intégration, mais ici on les
 applique et on les vérifie par le code.

[DÉFINITION – Formules essentielles]

CUBE de côté a :
 Volume = a^3
 Aire totale = $6 \cdot a^2$

PARALLÉLÉPIPÈDE rectangle (boîte) $a \times b \times c$:

$$\text{Volume} = a \cdot b \cdot c$$

$$\text{Aire totale} = 2 \cdot (ab + bc + ca)$$

SPHÈRE de rayon r :

$$\text{Volume} = \frac{4}{3} \pi r^3$$

$$\text{Aire (surface)} = 4 \pi r^2$$

CYLINDRE de rayon r et hauteur h :

$$\text{Volume} = \pi r^2 h$$

$$\text{Aire latérale} = 2 \pi r h$$

$$\text{Aire totale} = 2 \pi r (r + h)$$

CÔNE de rayon r , hauteur h , apothème $l = \sqrt{r^2 + h^2}$:

$$\text{Volume} = \frac{1}{3} \pi r^2 h$$

$$\text{Aire latérale} = \pi r l$$

$$\text{Aire totale} = \pi r (r + l)$$

PYRAMIDE à base B et hauteur h :

$$\text{Volume} = \frac{1}{3} B h$$

(généralisation du cône)

TÉTRAÈDRE régulier de côté a :

$$\text{Volume} = \frac{a^3}{6 \sqrt{2}}$$

$$\text{Aire totale} = \sqrt{3} a^2$$

[DÉMONSTRATION] Volume sphère = $\frac{4}{3} \pi r^3$ par la méthode de Cavalieri

On empile des disques horizontaux à la hauteur $z \in [-r, r]$.

Rayon du disque à hauteur z : $\rho = \sqrt{r^2 - z^2}$

Aire du disque : $\pi \rho^2 = \pi (r^2 - z^2)$

Volume = intégrale de $-r$ à r de $\pi (r^2 - z^2) dz$

$$= \pi \left[r^2 z - \frac{z^3}{3} \right] \text{ de } -r \text{ à } r$$

$$= \pi \left[\left(r^3 - \frac{r^3}{3} \right) - \left(-r^3 + \frac{r^3}{3} \right) \right]$$

$$= \pi \left[2r^3 - \frac{2r^3}{3} \right]$$

$$= \pi \left[\frac{4r^3}{3} \right]$$

$$= \frac{4}{3} \pi r^3 \quad \square$$

[CODE PYTHON]

```
class Sphere:
```

```
    def __init__(self, r):
```

```
        self.r = r
```

```
    def volume(self):
```

```
        return (4/3) * math.pi * self.r**3
```

```
    def aire(self):
```

```
        return 4 * math.pi * self.r**2
```

```
class Cylindre:
```

```
    def __init__(self, r, h):
```

```
        self.r = r
```

```
        self.h = h
```

```
    def volume(self):
```

```
        return math.pi * self.r**2 * self.h
```

```

def aire_laterale(self):
    return 2 * math.pi * self.r * self.h
def aire_totale(self):
    return 2 * math.pi * self.r * (self.r + self.h)

class Cone:
    def __init__(self, r, h):
        self.r = r
        self.h = h
        self.l = math.sqrt(r**2 + h**2) # apothème
    def volume(self):
        return (1/3) * math.pi * self.r**2 * self.h
    def aire_totale(self):
        return math.pi * self.r * (self.r + self.l)

class Parallelepipede:
    def __init__(self, a, b, c):
        self.a = a
        self.b = b
        self.c = c
    def volume(self):
        return self.a * self.b * self.c
    def aire_totale(self):
        a, b, c = self.a, self.b, self.c
        return 2 * (a*b + b*c + c*a)

# Tests
S = Sphere(1)
print("Sphère r=1 :")
print(f"  Volume = {S.volume():.6f}  (attendu : {4/3*math.pi:.6f}) ✓")
print(f"  Aire   = {S.aire():.6f}  (attendu : {4*math.pi:.6f}) ✓")

C = Cylindre(3, 5)
print(f"\nCylindre r=3, h=5 :")
print(f"  Volume = {C.volume():.4f}")
print(f"  Aire totale = {C.aire_totale():.4f}")

K = Cone(3, 4)
print(f"\nCône r=3, h=4 (apothème={K.l}) :")
print(f"  Volume = {K.volume():.4f}")

# Vérification de Cavalieri (approximation numérique du volume sphère)
def volume_sphere_cavalieri(r, n=10000):
    """Approximation par somme de disques"""
    dz = 2 * r / n
    total = 0
    for i in range(n):
        z = -r + (i + 0.5) * dz
        rho2 = r**2 - z**2
        if rho2 > 0:
            total += math.pi * rho2 * dz
    return total

```

```
print(f"\nVolume sphère r=1 par Cavalieri : {volume_sphere_cavalieri(1):.6f}")
print(f"Formule exacte : {(4/3)*math.pi:.6f}")
```

12. TRANSFORMATIONS GÉOMÉTRIQUES – déplacer et déformer

[THÉORIE]

Une transformation est une fonction qui envoie un point sur un autre.
Les transformations qui conservent les distances s'appellent ISOMÉTRIES (du grec : même mesure). Elles conservent les formes et les tailles.
En data science, elles sont omniprésentes dans : le traitement d'images, les rotations de caméra (vision par ordinateur), la réduction de dimension.

[DÉFINITION]

TRANSLATION de vecteur $v(a, b)$:

$$T(x, y) = (x + a, y + b)$$

ROTATION d'angle θ autour de $O(0,0)$:

$$R(x, y) = (x \cos(\theta) - y \sin(\theta), x \sin(\theta) + y \cos(\theta))$$

SYMÉTRIE AXIALE par rapport à l'axe x :

$$S(x, y) = (x, -y)$$

HOMOTHÉTIE de centre O , rapport k :

$$H(x, y) = (kx, ky)$$

Si $k > 1$: agrandissement

Si $0 < k < 1$: réduction

Si $k = -1$: symétrie centrale

En MATRICE (représentation compacte) :

Rotation : $\begin{bmatrix} \cos t & -\sin t \\ \sin t & \cos t \end{bmatrix}$

Symétrie x : $\begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$

Homothétie : $\begin{bmatrix} k & 0 \\ 0 & k \end{bmatrix}$

[DÉMONSTRATION] La composition de deux rotations d'angles θ_1 et θ_2 est une rotation d'angle $\theta_1 + \theta_2$.

$$\begin{aligned} R_1 * R_2 &= \begin{bmatrix} \cos(t_1) & -\sin(t_1) \\ \sin(t_1) & \cos(t_1) \end{bmatrix} \\ &\quad * \begin{bmatrix} \cos(t_2) & -\sin(t_2) \\ \sin(t_2) & \cos(t_2) \end{bmatrix} \\ &= \begin{bmatrix} \cos(t_1)\cos(t_2) - \sin(t_1)\sin(t_2) & -\cos(t_1)\sin(t_2) - \sin(t_1)\cos(t_2) \\ \sin(t_1)\cos(t_2) + \cos(t_1)\sin(t_2) & -\sin(t_1)\sin(t_2) + \cos(t_1)\cos(t_2) \end{bmatrix} \\ &= \begin{bmatrix} \cos(t_1+t_2) & -\sin(t_1+t_2) \\ \sin(t_1+t_2) & \cos(t_1+t_2) \end{bmatrix} \quad (\text{formules d'addition trigonométriques}) \\ &= R(t_1 + t_2) \quad \square \end{aligned}$$

[CODE PYTHON]

```
def translation(P, vx, vy):
    return Point2D(P.x + vx, P.y + vy)

def rotation(P, theta_deg):
```

```

"""Rotation autour de O d'angle theta (en degrés)"""
t = math.radians(theta_deg)
x = P.x * math.cos(t) - P.y * math.sin(t)
y = P.x * math.sin(t) + P.y * math.cos(t)
return Point2D(round(x, 9), round(y, 9))

def symetrie_axe_x(P):
    return Point2D(P.x, -P.y)

def homothetie(P, k):
    return Point2D(k * P.x, k * P.y)

P = Point2D(1, 0)

print("Point de départ :", P)
print("Translation (+2, +3) :", translation(P, 2, 3))    # (3, 3)
print("Rotation 90° :", rotation(P, 90))                # (0, 1)
print("Rotation 180° :", rotation(P, 180))              # (-1, 0)
print("Symétrie axe x :", symetrie_axe_x(Point2D(3, 4))) # (3, -4)
print("Homothétie k=2 :", homothetie(P, 2))             # (2, 0)

# Vérification : rotation 90° + rotation 90° = rotation 180°
P2 = rotation(rotation(P, 90), 90)
P3 = rotation(P, 180)
print(f"\nR90 o R90 = {P2} | R180 = {P3} → égaux : {abs(P2.x-P3.x)<1e-9 and abs(P2.y-P3.y)<1e-9} ✓")

# Transformation d'un triangle
def transformer_triangle(T, f):
    """Applique une transformation f à chaque sommet"""
    return Triangle(f(T.A), f(T.B), f(T.C))

A = Point2D(0, 0); B = Point2D(2, 0); C = Point2D(1, 2)
T_orig = Triangle(A, B, C)
T_rot = transformer_triangle(T_orig, lambda P: rotation(P, 45))

print(f"\nAire avant rotation : {T_orig.aire_heron():.4f}")
print(f"Aire après rotation : {T_rot.aire_heron():.4f}") # conservée ✓

```

[PIÈGE]

Les rotations 2D et 3D ne fonctionnent PAS pareil.

En 2D : une seule rotation (autour d'un point).

En 3D : on tourne autour d'un AXE (3 axes possibles : x, y, z).

Et les rotations 3D ne commutent PAS : R_x puis $R_y \neq R_y$ puis R_x .

C'est la source de bugs fréquents en vision par ordinateur et jeux vidéo.

13. TRIGONOMÉTRIE – angles et longueurs

[THÉORIE]

La trigonométrie est le lien entre les angles et les longueurs.
Dans un triangle rectangle, les côtés et l'angle sont liés par sin, cos, tan. Ces fonctions se généralisent ensuite au cercle unité, à toute la droite réelle, et permettent de modéliser tout phénomène périodique (son, signal, rotation...).

[DÉFINITION]

Dans un triangle rectangle d'hypoténuse h, côté adjacent a, côté opposé o :

$$\sin(\theta) = o / h$$

$$\cos(\theta) = a / h$$

$$\tan(\theta) = o / a = \sin(\theta) / \cos(\theta)$$

CERCLE UNITÉ (définition étendue) :

Le point M sur le cercle unité à l'angle θ a pour coordonnées :

$$M = (\cos(\theta), \sin(\theta))$$

IDENTITÉ FONDAMENTALE :

$$\cos^2(\theta) + \sin^2(\theta) = 1 \quad (\text{pour tout } \theta)$$

FORMULES D'ADDITION :

$$\cos(a + b) = \cos(a)\cos(b) - \sin(a)\sin(b)$$

$$\sin(a + b) = \sin(a)\cos(b) + \cos(a)\sin(b)$$

LOI DES SINUS (triangle quelconque ABC) :

$$a/\sin(A) = b/\sin(B) = c/\sin(C) = 2R$$

où R est le rayon du cercle circonscrit.

LOI DES COSINUS (généralisation de Pythagore) :

$$a^2 = b^2 + c^2 - 2bc\cos(A)$$

[DÉMONSTRATION] Identité fondamentale $\cos^2 + \sin^2 = 1$

M = (cos t, sin t) est sur le cercle unité (rayon 1, centre 0).

Distance de 0 à M : $\sqrt{\cos^2(t) + \sin^2(t)} = 1$ (rayon)

Donc $\cos^2(t) + \sin^2(t) = 1$ □

[CODE PYTHON]

```
import math
```

```
# Valeurs remarquables
```

```
angles_deg = [0, 30, 45, 60, 90, 120, 180, 270, 360]
```

```
print(f"{'Angle':>6} | {'cos':>8} | {'sin':>8} | {'tan':>10}")
```

```
print("-" * 45)
```

```
for deg in angles_deg:
```

```
    rad = math.radians(deg)
```

```
    c = round(math.cos(rad), 6)
```

```
    s = round(math.sin(rad), 6)
```

```
    t = round(math.tan(rad), 4) if abs(math.cos(rad)) > 1e-9 else "indéfini"
```

```
    print(f"{deg:>6}° | {c:>8} | {s:>8} | {str(t):>10}")
```

```
# Vérification identité fondamentale
```

```
print("\nVérification  $\cos^2 + \sin^2 = 1$  :")
```

```
for deg in range(0, 361, 15):
```

```

rad = math.radians(deg)
val = math.cos(rad)**2 + math.sin(rad)**2
assert abs(val - 1) < 1e-12, f"Erreur à {deg}°"
print("OK pour tous les angles de 0° à 360° (pas de 15°) ✓")

# Loi des cosinus – résoudre un triangle
def loi_cosinus(b, c, A_deg):
    """Calcule le côté a opposé à l'angle A"""
    A = math.radians(A_deg)
    return math.sqrt(b**2 + c**2 - 2*b*c*math.cos(A))

a = loi_cosinus(5, 7, 60)
print(f"\nTriangle b=5, c=7, A=60° → a = {a:.4f}") # ≈ 6.245

# Loi des sinus – trouver un angle
def angle_loi_sinus(a, b, B_deg):
    """Trouve l'angle A tel que a/sin(A) = b/sin(B)"""
    B = math.radians(B_deg)
    sin_A = a * math.sin(B) / b
    if abs(sin_A) > 1:
        return None # pas de solution
    return math.degrees(math.asin(sin_A))

A_deg = angle_loi_sinus(3, 5, 90)
print(f"Loi des sinus a=3, b=5, B=90° → A = {A_deg:.2f}°") # 36.87°

```

|| PARTIE 4 – MÉTHODES ET OUTILS TRANSVERSAUX ||

14. COORDONNÉES POLAIRES ET SPHÉRIQUES – changer de repère

[THÉORIE]

Le repère cartésien (x, y) n'est pas toujours le plus pratique.
 Pour décrire un cercle, il faut une équation compliquée en cartésien,
 mais triviale en polaire : $r = \text{constante}$.
 Même idée en 3D avec les sphériques (utiles en géographie, physique,
 traitement du signal...).

[DÉFINITION]

COORDONNÉES POLAIRES (2D) :

Tout point M repéré par (r, theta) où :
 r = distance à l'origine ($r \geq 0$)
 theta = angle avec l'axe x (en radians)

Conversion polaire → cartésien :

$x = r * \cos(\text{theta})$

```
y = r * sin(theta)
```

Conversion cartésien → polaire :

```
r = sqrt(x^2 + y^2)
```

```
theta = atan2(y, x)
```

COORDONNÉES SPHÉRIQUES (3D) :

Tout point M repéré par (r, theta, phi) où :

r = distance à l'origine

theta = angle polaire (colatitude, 0 à pi)

phi = azimut (0 à 2*pi)

Conversion sphérique → cartésien :

```
x = r * sin(theta) * cos(phi)
```

```
y = r * sin(theta) * sin(phi)
```

```
z = r * cos(theta)
```

[EXEMPLE]

M(r=2, theta=pi/3) en polaire → cartésien :

```
x = 2*cos(pi/3) = 2*0.5 = 1
```

```
y = 2*sin(pi/3) = 2*(sqrt(3)/2) = sqrt(3) ≈ 1.732
```

Cercle de rayon 3 : en cartésien $x^2+y^2=9$, en polaire $r=3$ (trivial!)

[CODE PYTHON]

```
def polaire_vers_cartesien(r, theta):
    return r * math.cos(theta), r * math.sin(theta)

def cartesien_vers_polaire(x, y):
    r = math.sqrt(x**2 + y**2)
    theta = math.atan2(y, x)
    return r, theta

def spherique_vers_cartesien(r, theta, phi):
    x = r * math.sin(theta) * math.cos(phi)
    y = r * math.sin(theta) * math.sin(phi)
    z = r * math.cos(theta)
    return x, y, z

# Test aller-retour polaire
r, theta = 2, math.pi / 3
x, y = polaire_vers_cartesien(r, theta)
r2, theta2 = cartesien_vers_polaire(x, y)
print(f"Polaire ({r}, {theta:.4f}) → Cartésien ({x:.4f}, {y:.4f})")
print(f"Retour → ({r2:.4f}, {theta2:.4f}) ✓")

# Dessiner un cercle en coordonnées polaires
print("\nPoints du cercle r=1 (polaire → cartésien) :")
for deg in range(0, 361, 45):
    theta = math.radians(deg)
    x, y = polaire_vers_cartesien(1, theta)
    print(f" theta={deg:3d}° → ({x:.4f}, {y:.4f})")
```

15. BARYCENTRE – le "centre de masse" géométrique

[DÉFINITION]

Le barycentre de n points $(A_1, A_2, \dots, A_n)$ affectés de coefficients $(w_1, w_2, \dots, w_n)$ avec $\sum(w_i) \neq 0$ est le point G tel que :

$$w_1 \cdot GA_1 + w_2 \cdot GA_2 + \dots + w_n \cdot GA_n = \text{vecteur } 0$$

En coordonnées :

$$G = \left(\frac{\sum(w_i \cdot x_i)}{\sum(w_i)}, \frac{\sum(w_i \cdot y_i)}{\sum(w_i)} \right)$$

Cas particuliers :

Milieu de AB : barycentre de $(A,1)$ et $(B,1)$

Centroïde d'un triangle : barycentre de $(A,1)$, $(B,1)$, $(C,1)$

$$G = ((x_A + x_B + x_C)/3, (y_A + y_B + y_C)/3)$$

[CODE PYTHON]

```
def barycentre(points_poids):
    """
    points_poids : liste de (Point2D, poids)
    Retourne le barycentre.
    """
    total_poids = sum(w for _, w in points_poids)
    if abs(total_poids) < 1e-12:
        raise ValueError("Somme des poids nulle : barycentre indéfini")
    x = sum(w * P.x for P, w in points_poids) / total_poids
    y = sum(w * P.y for P, w in points_poids) / total_poids
    return Point2D(x, y)

A = Point2D(0, 0); B = Point2D(6, 0); C = Point2D(3, 6)

# Centroïde du triangle (poids égaux)
G = barycentre([(A,1), (B,1), (C,1)])
print(f"Centroïde ABC = {G}") # Point(3.0, 2.0)

# Vérification : G = ((0+6+3)/3, (0+0+6)/3)
print(f"Formule directe = ({(0+6+3)/3}, {(0+0+6)/3})") # (3.0, 2.0) ✓

# Barycentre pondéré (centre de masse physique)
G2 = barycentre([(A, 2), (B, 5), (C, 1)])
print(f"Barycentre pondéré (2,5,1) = {G2}")

# Milieu comme cas particulier
I = barycentre([(A,1), (B,1)])
print(f"Milieu AB = {I}") # Point(3.0, 0.0) ✓
```

PLAN (2D)

Distance AB	=	$\sqrt{(x_B - x_A)^2 + (y_B - y_A)^2}$	
Milieu AB	=	$((x_A + x_B)/2, (y_A + y_B)/2)$	
Produit scalaire	=	$a \cdot c + b \cdot d$	
Angle entre u,v	=	$\arccos(u \cdot v / (u \cdot v))$	
Perpendicularité	:	$m_1 \cdot m_2 = -1$ (ou $u \cdot v = 0$)	
Droite	:	$ax + by + c = 0$	
Cercle	:	$(x-a)^2 + (y-b)^2 = r^2$	
Aire triangle	=	$(1/2) \cdot \text{base} \cdot \text{hauteur}$	
	=	$\sqrt{s(s-a)(s-b)(s-c)}$ [Héron]	
Centroïde	=	$((x_A + x_B + x_C)/3, (y_A + y_B + y_C)/3)$	
Rotation	:	$x' = x \cdot \cos(t) - y \cdot \sin(t)$	
		$y' = x \cdot \sin(t) + y \cdot \cos(t)$	

ESPACE (3D)

Distance AB	=	$\sqrt{(x_B - x_A)^2 + (y_B - y_A)^2 + (z_B - z_A)^2}$	
Produit scalaire	=	$ad + be + cf$	
Produit vectoriel	=	$(bf - ce, cd - af, ae - bd)$	
Plan	:	$ax + by + cz + d = 0$	
Normale	:	vecteur (a, b, c) dans $ax + by + cz + d = 0$	
Dist. point-plan	=	$ ax_0 + by_0 + cz_0 + d / \sqrt{a^2 + b^2 + c^2}$	
Droite	:	$M = A + t \cdot d$ (paramétrique)	
Sphère	:	$(x-a)^2 + (y-b)^2 + (z-c)^2 = r^2$	
Volume sphère	=	$(4/3) \cdot \pi \cdot r^3$	
Aire sphère	=	$4 \cdot \pi \cdot r^2$	
Volume cylindre	=	$\pi \cdot r^2 \cdot h$	
Volume cône	=	$(1/3) \cdot \pi \cdot r^2 \cdot h$	
Volume pyramide	=	$(1/3) \cdot \text{base} \cdot h$	

TRIGONOMÉTRIE

$\sin^2 + \cos^2 = 1$	(identité fondamentale)	
$\tan$	$= \sin / \cos$	
Loi des cosinus	: $a^2 = b^2 + c^2 - 2bc \cdot \cos(A)$	
Loi des sinus	: $a / \sin(A) = b / \sin(B) = c / \sin(C) = 2R$	
$\sin(a+b)$	$= \sin(a)\cos(b) + \cos(a)\sin(b)$	
$\cos(a+b)$	$= \cos(a)\cos(b) - \sin(a)\sin(b)$	
Valeurs clés	: $\sin(30^\circ) = 1/2, \sin(45^\circ) = \sqrt{2}/2, \sin(60^\circ) = \sqrt{3}/2$	

#####

#

CE QUE TU MAÎTRISES MAINTENANT :

#

✓ Points et repères – coordonnées cartésiennes 2D et 3D

```
# ✓ Vecteurs – déplacement, norme, Chasles
# ✓ Produit scalaire – angle, perpendicularité, projection
# ✓ Produit vectoriel – normale, aire, colinéarité (3D)
# ✓ Distance / Milieu – formules et vérifications
# ✓ Droites 2D – cartésienne, réduite, paramétrique
# ✓ Triangles – aire (Héron, vecteur), angles, classification
# ✓ Cercle – équation, tangente, intersection
# ✓ Plans 3D – équation, normale, distance point-plan
# ✓ Droites 3D – paramétrique, droites gauches
# ✓ Solides – volumes et aires (sphère, cylindre, cône, pyramide)
# ✓ Transformations – translation, rotation, symétrie, homothétie
# ✓ Trigonométrie – sin/cos/tan, loi des cosinus, loi des sinus
# ✓ Coordonnées polaires et sphériques – changement de repère
# ✓ Barycentre – centroïde, milieu, centre de masse
#
# PROCHAINE ÉTAPE NATURELLE :
# → Algèbre linéaire (matrices, déterminants, valeurs propres)
# → Analyse (courbes paramétriques, intégrales pour volumes/aires)
# → Géométrie différentielle (courbure, surfaces)
# → Géométrie analytique avancée (coniques : ellipse, parabole, hyperbole)
#
#####
```